



CortexAI Redux State Management Documentation

This document explains the variables, functions, and the detailed file-to-file state flow showing how Redux manages and distributes data across CortexAI.

1. Setup & Configuration (The Foundation)

A. Providing the Store

- **File:** `main.jsx`
- **What it does:** Wraps the React application in `<Provider store={store}>`.
- **How it works:** Makes the global state store accessible to all components in the React tree.

B. Configuring the Store

- **File:** `store.js`
 - **What it does:** Aggregates separate state slices into a single unified store.
 - **How it works:** Combines the reducers from `userslice.js` (mapped under key `user`) and `conversationslice.js` (mapped under key `conversation`).
-

2. Slices, Variables, & Actions Dictionary

A. User Slice (`userslice.js`)

- **State Variable:**
 - `userData` (Initial: `null`): Stores the database record of the logged-in user.
- **Action / Reducer:**
 - `setUserdata(payload)`: Sets the profile details of the user or clears it (`null`) on

logout.

B. Conversation Slice (`conversationslice.js`)

- **State Variables:**
 - `conversations` (Initial: `[]`): Array containing the history of chat sessions.
 - `selectedConversation` (Initial: `null`): Holds the conversation object currently active in the chat area.
 - **Actions / Reducers:**
 - `setConversations(payload)` : Overwrites the list with chats fetched from MongoDB.
 - `addConversation(payload)` : Prepends a new chat session to the beginning of the list.
 - `setSelectedConversation(payload)` : Updates the store with the chat session currently chosen by the user.
-

3. Global Redux Integration (Where & How Data is Consumed)

Redux acts as the single source of truth. Below is where and why each slice is used across the application:

A. Authentication Data (`userData`)

- **In `App.jsx` (Auto-Login Sync):**
 - *Where:* Mounted inside the main React shell.
 - *Why:* Restores the user session on page refresh.
 - *How:* Fires `getCurrentuser()` from features and dispatches `dispatch(setUserdata(data))` to load the profile.
- **In `home.jsx` (Conditional Router):**
 - *Where:* Main workspace router shell.
 - *Why:* Checks if the user is authenticated. If `userData` is `null`, it renders a login popup modal; once `userData` gets populated, it mounts the home screen and sidebar.

- *How:* Reads data using `useSelector((state) => state.user.userData)`.
 - In `sidebar.jsx` (**Profile & Logout Panel**):
 - *Where:* Sidebar component.
 - *Why:* Reads profile info to display name, email, avatar image, and check credits.
 - *How:* Extracts user details from Redux. On clicking the logout button, it dispatches `dispatch(setUserdata(null))` to trigger the login screen immediately.
-

B. Conversation Data (`conversations` & `selectedConversation`)

- In `sidebar.jsx` (**List & Highlighting**):
 - *Where:* Conversation history container.
 - *Why:* Displays the user's past chats and highlights the active one.
 - *How:*
 - Reads `conversations` and `selectedConversation` using `useSelector`.
 - Renders the list via `conversations.map()`.
 - Highlights the selected chat by checking `selectedConversation?._id === conv?._id`.
 - Dispatches `setSelectedConversation(conv)` when a chat list item is clicked.
 - Dispatches `addConversation(newChat)` when creating a new chat to instantly append it to the sidebar list.
 - In `chatarea.jsx` (**Active Chat Interface**):
 - *Where:* Chat messaging container.
 - *Why:* Restores and displays message bubbles for the active chat session.
 - *How:* Reads `selectedConversation` to identify which chat history to query and render.
-

4. File-to-File State Execution Workflow

Here is how data executes across files when performing actions:

Workflow 1: Google Login Flow

```
[User Action] home.jsx
|   —> User logs in via Google and backend sends user object back.
|   —> Triggers: dispatch(setUserdata(userObject))
▼
[State Manager] redux/userslice.js
|   —> Action triggers reducer: state.userData = action.payload
▼
[Listener] home.jsx
|   —> useSelector detects state.user.userData is no longer null.
|   —> Re-renders UI: removes login modal and displays <SideBar /> and <ChatArea />.
```

Workflow 2: Select Conversation Flow

```
[User Action] sidebar.jsx
|   —> User clicks on a past conversation bubble in the sidebar list.
|   —> Triggers: dispatch(setSelectedConversation(clickedConvObject))
▼
[State Manager] redux/conversationslice.js
|   —> Action triggers reducer: state.selectedConversation = action.payload
▼
[Listener] chatarea.jsx
|   —> useSelector detects selectedConversation._id has changed.
|   —> Fires API: fetches all messages associated with that active ID.
|   —> Re-renders UI: displays the message history of the selected chat.
```